

	Pimpri Chinchwad Education Trust's Pimpri Chinchwad College of Engineering (PCCoE) (An Autonomous Institute) Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV	
Full Stack Development Laboratory		

Assignment-4

Name – Mukund Fegade

PRN-123B1B197

Batch – C3 FDSL

Assignment 4

Title:

Create an Interactive Data Visualization Dashboard for a Real-Life Application (e.g., Weather Dashboard) using Charts and Graphs (Chart.js / D3.js)

Objectives

- To understand the concept of data visualization in web applications.
- To design an interactive dashboard for real-life data representation.
- To visualize data using charts and graphs with libraries such as Chart.js or D3.js.
- To learn how JavaScript integrates with visualization libraries.
- To present complex data in an easy-to-understand graphical format.

Outcomes

- Students will be able to create interactive dashboards using visualization libraries.
- Students will understand how graphical representation improves data analysis.
- Students will gain experience integrating external JavaScript libraries into web applications.
- Students will learn how to display real-world datasets using charts such as bar, line, and pie charts.

Theory Concept

A Data Visualization Dashboard is a web-based interface that displays information visually using charts, graphs, and indicators. Instead of viewing raw numerical data, users can easily analyze trends, comparisons, and patterns through graphical representation.

1. Data Visualization

Data visualization is the process of representing data using visual elements such as charts, graphs, and maps. It helps users quickly understand large datasets and make informed decisions.

Benefits of data visualization:

- Easy interpretation of complex data
- Quick identification of trends and patterns

- Improved decision-making
 - Enhanced presentation quality
-

2. HTML (Structure Layer)

HTML is used to create the layout of the dashboard, including:

- Header section
 - Chart containers
 - Data panels
-
- Titles and labels
 - Dashboard layout structure

It provides the foundation and structural elements of the webpage.

3. CSS & Bootstrap (Styling and Responsiveness)

CSS and Bootstrap are used to design visually appealing dashboards.

Their roles include:

- Styling fonts and colors
- Creating responsive grid layouts
- Designing cards and panels
- Ensuring mobile-friendly interface

Bootstrap provides pre-designed components and layout utilities that improve development speed and consistency.

4. JavaScript (Interactivity Layer)

JavaScript adds dynamic behavior to the dashboard.

Key functionalities include:

- Loading datasets
- Updating charts dynamically
- Handling user interactions
- Changing datasets based on selection
- Rendering charts without page reload

JavaScript connects data with visualization libraries to create an interactive user experience.

5. Chart.js / D3.js Libraries

Visualization libraries simplify the process of creating charts and graphs.

Chart.js

- Easy-to-use JavaScript library
- Supports bar, line, pie, doughnut, and radar charts
- Automatically responsive
- Suitable for beginners

D3.js

- Advanced data visualization library
- Allows highly customizable and interactive graphics
- Suitable for complex visualizations

These libraries convert numerical data into graphical formats using JavaScript.

6. Real-Life Application – Weather Dashboard

A Weather Dashboard visually represents weather-related data such as:

- Temperature trends
- Humidity levels
- Rainfall statistics
- Weekly forecasts
- Wind speed

Charts help users analyze weather patterns easily instead of reading raw numerical values. Graphical representation improves clarity and understanding.

Dataset Used

The dashboard uses sample or real-world weather data, which may include:

- Temperature readings (daily or weekly)
- Humidity percentage
- Wind speed
- Rainfall data
- Weather conditions

The data may be stored in JavaScript arrays/objects or fetched from a weather API. For this experiment, sample static data can be used to simulate real-world visualization.

Index.html –

```
<!DOCTYPE html>

<html>

<head>

  <title>Simple Weather Dashboard</title>

  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

  <style>

    body {

      font-family: Arial;

      text-align: center;

      background-color: #f4f6f9;

    }

    h1 {

      margin-top: 20px;

    }

    .chart-container {

      width: 70%;

      margin: 20px auto;

    }

  </style>

</head>

<body>

  <h1>Simple Weather Dashboard</h1>

  <div class="chart-container">

    <img alt="Line chart showing temperature trends over time." data-bbox="113 463 629 963"/>

  </div>

</body>

</html>
```

```
    button {
        padding: 8px 15px;
        margin: 10px;
        cursor: pointer;
    }
</style>
</head>
<body>

    <h1>Weather Dashboard</h1>

    <button onclick="updateData()">Change Week Data</button>

    <div class="chart-container">
        <canvas id="tempChart"></canvas>
    </div>

    <div class="chart-container">
        <canvas id="humidityChart"></canvas>
    </div>

<script>

    const days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];

    let temperatureData = [30, 32, 31, 29, 35, 36, 34];
    let humidityData = [60, 65, 55, 70, 75, 68, 72];

    const tempChart = new Chart(document.getElementById("tempChart"), {
```

```

type: "line",
data: {
  labels: days,
  datasets: [{
    label: "Temperature (°C)",
    data: temperatureData,
    borderColor: "blue",
    fill: false, tension:
    0.3
  }]
}
});

```

```

const humidityChart = new
Chart(document.getElementById("humidityChart"), { type:
  "bar",
  data: {
    labels: days,
    datasets: [{
      label: "Humidity (%)", data:
      humidityData,
      backgroundColor: "green"
    }]
  }
});

```

```

function updateData() {

```

```

temperatureData = temperatureData.map(() =>
  Math.floor(Math.random() * 10) + 28
);

humidityData = humidityData.map(() =>
  Math.floor(Math.random() * 20) + 50
);

tempChart.data.datasets[0].data = temperatureData;
humidityChart.data.datasets[0].data = humidityData;

tempChart.update();
humidityChart.update();
}
</script>

</body>
</html>

```

WebPage -



